

Module 5: Mining Data Streams

This module introduces practical techniques for sampling, summarizing, and analyzing unbounded data streams in real time.

Introduction to Mining Data Streams

Welcome to Module 5! This module introduces practical techniques for working with data streams, where data arrives continuously and cannot be stored or processed all at once. We'll explore key methods for sampling, estimating distinct values, and summarizing recent trends in real time. Using tools like reservoir sampling, HyperLogLog, and exponential decay, along with Spark Structured Streaming, you'll learn how to handle unbounded data efficiently and build simple, scalable streaming pipelines.

Learning Objectives

By the end of this module, you will be able to:

- Explain the constraints and characteristics of the stream data model.
- Apply sampling and summarization techniques to streaming data.
- Build a simple streaming analytics pipeline using Spark.

Readings and Resources

Please complete the following readings by the end of the week:

[Mining of Massive Datasets](#)

[Structured Streaming Programming Guide Assessments](#)

Assessments

For this module, you will need to complete the following:

[Individual Quiz 1 \(10%\)](#) , which is due in Week 5.

The Stream Data Model

In traditional data processing, we usually work **with static datasets**; data that can be stored, read multiple times, and processed as a whole. But in many real-world

systems, data comes in as a stream; a continuous, potentially infinite flow of items such as logs, sensor readings, user clicks, transactions, etc.

In a data stream, we don't have the luxury of storing everything. Instead, we need to process each incoming item on the fly, often with limited memory and only one pass.

So, the stream data model is defined by:

Unbounded Input,

Single-Pass Algorithms,

Memory-Limited,

Timeliness.

Unbounded Input

The data keeps arriving.



Single-Pass Algorithms

We can't go back to revisit old data.



Memory-Limited

We can't store the whole stream.



Timeliness

We care about recent or current state, not just historical aggregates.



Simple Python Example

Here's how we can simulate a basic stream using Python generators:

```
import time
```

```
import random
def stream_generator():
    while True:
        yield {"timestamp": time.time(), "value": random.randint(0, 100)}
stream = stream_generator()
for _ in range(5):
    print(next(stream))
    time.sleep(1)
```

This simulates an unbounded stream of data, here just random numbers, arriving one at a time. You could replace this with anything: incoming web logs, sensor readings, etc.

Types of Queries on Streams

Because we can't store everything, most queries on streams fall into summary categories:

Query Type	Example	Goal
Aggregate	Average temperature in last 5 minutes	Summarize values
Frequency	How often "error" appears in logs	Track how common events are
Uniqueness	Count of distinct users in real-time	Estimate cardinality

We'll see techniques later in the module for these.

Window Models

Since we can't store or process an entire infinite stream, we focus on windows which are chunks of the stream that we can handle. These windows define what subset of data is currently "relevant" for our computations.

Sliding Window

A sliding window holds the most recent N items or T seconds of data, continuously moving forward. It's useful when we want metrics that are always updated based on the latest information. As an example of a use-case for sliding window, you can imagine counting how many logins happened in the last 60 seconds, updating every 5 seconds.

```
from collections import deque
```

```
import random
import time
window = deque(maxlen=10) # Holds last 10 items
for _ in range(20):
    value = random.randint(0, 100)
    window.append(value)
    print("Current window:", list(window))
    time.sleep(0.5) # Simulates time delay in stream
```

This code keeps only the most recent 10 values from a simulated stream. If you run it, you'll see the window shifting as new items arrive and old ones drop off.

Tumbling Window

A tumbling window breaks the stream into **non-overlapping** fixed-size chunks. Each event belongs to one and only one window.

This is useful for **periodic batch-style** processing of streams (e.g., per minute, per hour) such as calculating average temperature for each 1-minute interval.

```
def tumbling_window(stream, window_size=5):
    buffer = []
    for item in stream:
        buffer.append(item)
        if len(buffer) == window_size:
            print("Window batch:", buffer)
            buffer.clear()
# Simulated stream
stream = (random.randint(0, 50) for _ in range(20))
tumbling_window(stream, window_size=5)
```

This code groups incoming data into separate, non-overlapping chunks of 5 items. Give it a try and you'll notice how each batch is processed independently once it fills up.

Decaying Window (Exponential Decay)

A decaying window doesn't discard older data abruptly. Instead, it **weights recent data more heavily** and gives older data less importance over time. This is often more natural for detecting trends or changes. The weight decreases exponentially with age.

Formula (just for intuition):

$$\text{Weight} = \exp(-\lambda \times \text{age})$$

```
import math
import time
def exponential_decay(values, timestamps, lambda_=0.1):
    now = time.time()
    weighted_sum = 0
    for v, t in zip(values, timestamps):
        age = now - t
        weight = math.exp(-lambda_ * age)
        weighted_sum += v * weight
    return weighted_sum
# Simulated time series
timestamps = [time.time() - i for i in range(5)] # 5s ago to now
values = [10, 20, 30, 40, 50]
print("Exponentially weighted sum:", exponential_decay(values, timestamps))
```

This code calculates a weighted sum where recent values count more than older ones. Running it will show how the decay function emphasizes newer data while still considering the past.

These concepts form the backbone of how we process and summarize data streams. In the next sections, we'll apply them using sampling and sketching techniques.

Sampling in Streams

In data streams, we often don't have the option to store everything. The stream might be infinite, and even if it's not, storing or processing every single data point can be too expensive. This is where **sampling** comes in.

Sampling gives us a practical way to keep a small, manageable subset of the stream that still (hopefully) reflects the overall data.

Sampling is useful when we want to:

Get a quick summary without full aggregation.

Monitor trends or anomalies without full logs.

Feed downstream systems that can't handle the full stream.

In batch processing, you'd just take a random sample using built-in functions. In streaming, it's trickier:

We don't know the full size ahead of time.

We only see each item once.

So, we need specialized techniques that can work incrementally and with limited memory.

Reservoir Sampling

Reservoir Sampling is designed to **uniformly sample k items** from a stream of unknown or infinite length. It guarantees that, at any point, each item seen so far has an equal chance of being in the sample.

The algorithm works as follows:

Initialize a reservoir (list) with the first k items from the stream.

For each subsequent item at position i (starting from $i = k$):

Pick a random integer j between 0 and i.

If $j < k$, replace the item at position j in the reservoir with the new item.

This is surprisingly efficient and easy to implement. It only stores k items, no matter how long the stream gets, and it doesn't require any knowledge of how many items are coming.

Example

Let's walk through it with the Python implementation of it.

Here's a working Python function that samples 5 items from a stream of 100 integers:

```
import random
def reservoir_sample(stream, k):
    reservoir = []
    for i, item in enumerate(stream):
        if i < k:
            reservoir.append(item)
```

```
    else:
        j = random.randint(0, i)
        if j < k:
            reservoir[j] = item
    return reservoir
# Simulated stream: numbers 1 to 100
stream = range(1, 101)
sample = reservoir_sample(stream, 5)
print("Reservoir sample:", sample)
```

Try running this a few times. You'll see a different output each time, but the items will always be chosen uniformly from the full range, even though we're never storing more than 5 at once.

This method is extremely useful when you want to capture a “**representative snapshot**” of the stream at any point.

Counting Distinct Elements

One of the most common questions we ask in data analysis is:

How many unique things have we seen so far?

This comes up in use cases like:

Examples

Counting unique users hitting an API.

Tracking distinct IP addresses in network logs.

Estimating the number of unique hashtags used on a social platform.

In a batch context, we'd probably:

Just throw everything into a `set()`,

Use `COUNT(DISTINCT ...)` in SQL,

But with streaming data, this approach doesn't scale. Storing all unique values in memory works fine for small datasets, but it breaks down quickly when the number of distinct items grows, especially when we don't know how big the stream will be.

Naive Approach: Using a Set

Just to show what the simple solution looks like, here's the naive method in Python:

```
import random
def count_distinct_naive(stream):
    seen = set()
    for item in stream:
        seen.add(item)
    return len(seen)
# Example stream
stream = [random.randint(1, 1000) for _ in range(10000)]
print("Naive distinct count:", count_distinct_naive(stream))
```

This works by storing every unique item in a set, and then counting the size of the set at the end.

If you run this code, you'll almost always see the output count of 1000. This happens because we're generating 10,000 random integers from only 1,000 possible values. With that many samples, it's highly likely that all values between 1 and 1000 will be hit at least once. So the set ends up containing exactly 1000 elements.

But the important thing to notice is this: we're storing every unique value in memory. That may be fine when we're only dealing with 1,000 distinct items, but imagine doing this for millions of user IDs or IPs. The memory usage grows linearly with the number of unique elements, which becomes a problem at scale.

The Practical Alternative: Probabilistic Counting

To deal with this, we can use probabilistic algorithms that estimate the number of distinct elements using much less memory, with acceptable error margins. These methods don't give exact counts, but they're often good enough for real-time analytics, monitoring, or anomaly detection. One of the most widely used of these is **HyperLogLog**.

HyperLogLog: How It Works (Intuition)

HyperLogLog (HLL) is a smart algorithm for estimating cardinality (i.e. the number of distinct elements). Here's the basic idea:

Every item is hashed to a random-looking bit string.

The algorithm tracks how many leading zeros appear in the hash.

More leading zeros = rarer event = likely more unique values.

It does this across many small buckets and combines the results to estimate the total.

The key advantages:

Uses fixed-size memory, even as the stream grows.

Works well with large-scale data.

Provides statistically bounded error (which you can control via a precision parameter).

We'll use the datasketch library to try it out in Python.

HyperLogLog in Python with datasketch

If you haven't installed datasketch yet on your computer, you can do so by:

pip install datasketch

Then, here's how to use HyperLogLog on a stream of integers:

```
from datasketch import HyperLogLog
import random
# Initialize HLL with precision p (higher = more accurate, more memory)
hll = HyperLogLog(p=10)
# Simulated stream with many duplicates
stream = [random.randint(1, 10000) for _ in range(50000)]
for item in stream:
    hll.update(str(item).encode('utf-8')) # Input must be bytes
print("HyperLogLog estimate:", int(hll.count()))
```

If you run this code, you'll typically get something like:

HyperLogLog estimate: 10105

You won't necessarily get exactly 10,000, even though the stream generates numbers between 1 and 10,000. This is because some values might not appear in just 50,000 random draws.

HyperLogLog will usually estimate a value close to or slightly higher than the true number of unique items, especially with a lower precision setting like $p=10$. That's expected as it trades off a small amount of accuracy for memory efficiency.

Comparing to the Naive Approach

If we wanted to compare HyperLogLog with the naive method fairly, we'd need to use the same stream. But even when using different streams, we can still look at how the two methods behave and what trade-offs they involve.

Here's what that might look like with the HLL stream:

```
# Naive count (exact)
true_distinct = len(set(stream))
# HyperLogLog estimate
hll_estimate = int(hll.count())
print("True distinct count:", true_distinct)
print("HyperLogLog estimate:", hll_estimate)
print("Relative error:", abs(hll_estimate - true_distinct) / true_distinct)
```

If you run this, expect something like:

True distinct count: 9919

HyperLogLog estimate: 10082

Relative error: 0.01643310817622744

The exact numbers will vary depending on the random stream, but in general:

The **naive method** gives an **exact** count but stores all unique items.

HyperLogLog gives an **approximate** count using much less memory.

This makes HLL a great fit for large, high-cardinality streams where space is limited, and perfect accuracy isn't required.

Spark's `approx_count_distinct`

Using Spark, there's a built-in way to get a distinct count estimate using the same idea:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import rand
# Start Spark session
spark = SparkSession.builder.appName("DistinctExample").getOrCreate()
# Create DataFrame with user_id column of random integers between 0 and 9999
df = spark.range(0, 100000).withColumn("user_id", (rand() * 10000).cast("int"))
# Approximate distinct count using HyperLogLog under the hood
df.selectExpr("approx_count_distinct(user_id)").show()

```

In the above code, `spark.range(0, 100000)` creates a DataFrame with 100,000 rows. `rand() * 10000` generates random numbers between 0 and 9999, and `cast("int")` turns them into integers. The `approx_count_distinct(user_id)` function then estimates how many unique user IDs there are using HyperLogLog.

When you run this, you might not get exactly 10,000 because:

The values are generated randomly, so some user IDs may repeat.

Spark's `approx_count_distinct` uses HyperLogLog, so the result is an **estimate**, not an exact count.

Depending on the precision Spark uses (which varies by context), the result will typically be within 1–2% of the actual distinct count, just like we saw with the Python `datasketch` example.

Summary of Stream Data Over Time

In streaming analytics, we often care about **what's happening recently**, rather than over the entire stream. Whether we're tracking live metrics, detecting changes, or responding to sudden spikes, we need methods that let us summarize the latest part of the stream efficiently.

You've already seen how window models like sliding, tumbling, and exponential decay work in concept. Next, we'll focus more closely on how to use these, especially exponential decay, to summarize data streams in practice.

Rolling Averages with Fixed-Size Windows

One of the simplest but most useful summaries is a rolling average. This helps smooth out short-term fluctuations and gives a sense of recent behavior. It's often used in monitoring tools, real-time dashboards, and basic anomaly detection.

Here's a basic implementation using a fixed-size window in Python:

```
from collections import deque
import random
import time
window = deque(maxlen=5)
for _ in range(15):
    value = random.randint(0, 100)
    window.append(value)
    avg = sum(window) / len(window)
    print(f'New value: {value}, Rolling average: {avg:.2f}')
    time.sleep(0.5)
```

This simulates a stream of random values and maintains a moving window of the 5 most recent. As new values arrive, old ones drop off, and the average updates. It's simple, but gives you a feel for recent trends without needing to process or store the entire stream.

Exponential Decay for Real-Time Trend Detection

While sliding windows give a hard cut-off (e.g., “last 5 items”), exponential decay gives a smoother, more memory-efficient way to prioritize recent values.

This is especially useful when:

You want to respond quickly to recent changes in the stream.

You want to give partial influence to older data rather than dropping it completely.

You need to keep memory usage minimal; no need to store full windows.

We already introduced the idea of exponential decay. Now let's implement it more carefully, not just as a weighted sum, but as a **normalized weighted average**:

```

import math
import time
def exponential_decay(values, timestamps, lambda_=0.1):
    now = time.time()
    weighted_sum = 0
    total_weight = 0
    for v, t in zip(values, timestamps):
        age = now - t
        weight = math.exp(-lambda_ * age)
        weighted_sum += v * weight
        total_weight += weight
    return weighted_sum / total_weight if total_weight > 0 else 0

```

This function takes in a list of values and their corresponding timestamps and applies an exponentially decreasing weight to each value based on its age. Then, it returns the **decay-weighted average**, which emphasizes more recent data.

Let's try it with some fake data:

```

now = time.time()
timestamps = [now - i for i in reversed(range(5))] # ensures correct order: oldest to newest
values = [10, 20, 30, 150, 200]
print("λ=0.1:", exponential_decay(values, timestamps, lambda_=0.1))
print("λ=0.3:", exponential_decay(values, timestamps, lambda_=0.3))
print("λ=0.7:", exponential_decay(values, timestamps, lambda_=0.7))

```

Here, the values increase over time, with the most recent items (150, 200) being much larger than the older ones. As you increase λ , older values are down-weighted more heavily, and the decayed average shifts closer to the most recent values.

You should see an output like this:

$\lambda = 0.1$: 92.34

$\lambda = 0.3$: 113.01

$\lambda = 0.7$: 147.91

This demonstrates the effect of exponential decay clearly: with smaller λ values, older data still contributes significantly to the average, resulting in a more balanced influence across time.

As lambda increases, the decay becomes steeper, and the average becomes dominated by the most recent, higher values. This approach adapts smoothly to changing data and is especially useful for capturing real-time trends without the need to store or process entire windows of historical data.

Rolling vs. Decaying

The following identifies when to use rolling windows versus exponential decay:

Use Rolling Windows:

For fixed-size summaries, especially when matching a specific recent window (like last 5 minutes).

Use Exponential Decay:

For decay for smoother trend detection without storing full windows. Rolling gives hard cutoffs; decay gives a continuous blend.

Building a Simple Streaming Pipeline in Spark

To wrap up the module, let's implement a simple streaming analytics pipeline using **Spark Structured Streaming**. The goal here is to demonstrate how the core ideas we've explored, like sampling, windowing, and summarization, can be used in a real-time setting.

Before we dive into the code, it helps to understand a couple of key concepts behind how Spark Structured Streaming works under the hood.

Streaming as an Unbounded Table

In Spark, a stream is treated conceptually as an unbounded table. Each new event that arrives is like a new row being appended to a table that never stops growing. Spark periodically takes snapshots of this evolving table (called micro-batches), applies transformations, and produces results at each trigger interval. This model lets you use familiar DataFrame operations while working with continuous data.

Output Modes in Structured Streaming

When writing the results of a streaming query, you need to choose an **output mode**. This defines what part of the result Spark writes after each batch:

Append Mode

Only new rows added since the last trigger are written. Used when events are added but not updated (e.g., log append).

Update Mode

Only rows that changed since the last trigger are written. Best for stateful aggregations like counts or averages.

Complete Mode

The entire result table is written every time. Useful for debugging or small datasets.

In the following example, we're using `.outputMode("complete")`, but you can change it to "update" or "append" depending on the logic and sink used.

Spark Structured Streaming Example: A Simple Pipeline

You are now invited to participate in the next example. Please read the benefits before entering the code into Spark.

In this example, we'll:

Ingest data from a built-in source (rate),

Generate synthetic values to simulate a live metric (e.g., temperature, load, etc.),

Apply a windowed aggregation,

Use `approx_count_distinct` as a probabilistic summarization step,

Output the results to the console.

Here is the code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import expr, rand, window
# Start Spark session
spark = SparkSession.builder.appName("StreamPipeline").getOrCreate()
# Simulated stream: one row per second
stream = (
    spark.readStream
```

```

        .format("rate")
        .option("rowsPerSecond", 5)
        .load()
    )
    # Simulate a 'value' column to work with
    data = stream.withColumn("value", (rand(seed=None) * 100).cast("int"))
    # Add a synthetic 'user_id' to demonstrate distinct count
    data = data.withColumn("user_id", (rand(seed=None) * 1000).cast("int"))
    # Define a 10-second tumbling window
    aggregated = (
        data.groupBy(window("timestamp", "10 seconds"))
        .agg(
            expr("avg(value) as avg_value"),
            expr("approx_count_distinct(user_id) as est_unique_users")
        )
    )
    # Output the result to the console
    query = (
        aggregated.writeStream
        .outputMode("complete") # try "update" or "append" as alternatives
        .format("console")
        .option("truncate", False)
        .start()
    )
    query.awaitTermination()

```

This pipeline shows how to connect ingestion, transformation, and summarization in a live setting using Spark.

When you run this Spark Structured Streaming pipeline, the output will appear continuously in the console, grouped by batch number (e.g., Batch: 0, Batch: 1, and so on). Each batch contains aggregated results for a 10-second time window, showing the average value and an estimated count of distinct user IDs seen during that period.

By default, the stream keeps running **indefinitely** until you manually stop it (e.g., with Ctrl+C or by interrupting the cell). If you're running this in a notebook environment, make sure to interrupt the cell execution, and then run `spark.stop()` in a separate cell to fully shut down the Spark session.

If you change the output mode to "update", Spark will print only the rows that changed in the current batch. If you use "append", Spark will only output completed windows which are the rows that are finalized and will not be updated again. Each mode affects both what you see and when results appear.

Summary of Introduction to Mining Data Streams

In this module, we explored practical methods for processing streaming data, including reservoir sampling, HyperLogLog for distinct counting, and summarizing trends with sliding windows and exponential decay. These techniques help manage unbounded streams efficiently under memory and time constraints. We concluded with a hands-on Spark Structured Streaming pipeline, showing how real-time data can be ingested, transformed, and summarized in a scalable, end-to-end system.

In summary, we learned how to:

Explain the constraints and characteristics of the stream data model.

Apply sampling and summarization techniques to streaming data.

Build a simple streaming analytics pipeline using Spark.